Fault-Tolerant Topic-Based File Distribution Service

IT305 Computer Networks Laboratory — Final Implementation & Performance Evaluation Report

Institution: Dhirubhai Ambani University (Formerly DA-IICT), Gandhinagar | **Course:** IT305 — Computer Networks (Lab Assignment 6, Lab G4, Autumn 2026)
Merged Project Group: Group 071 + Group 072 | **Submission Date:** 16 October 2026

Group Members & Individual Contributions

The project was executed using a strictly sequential single-owner engineering chain across six phases, where each member took end-to-end ownership of a core subsystem and passed a formal verification exit gate before handing off to the next phase:

Phase	Member Name	Roll Number	One-Line Summary of Individual Contribution
Phase 1	**Jay Vaghela**	`202401237`	Implemented the 16-byte binary wire framing protocol (`common/protocol.*`), path-sanitized CRC32 topic manifest engine (`common/topic_manager.*`), and the Part 1 Sequential & Multi-Threaded server (`Part1/server.c`).
Phase 2	**Vivek Dodiya**	`202401253`	Implemented the bounded 64 KiB file streaming and disk durability engine (`common/file_transfer.*`), recursive directory creation, and the Part 1 baseline client (`Part1/client.c`) with end-to-end CRC32 integrity verification.
Phase 3	**Atharv Trivedi**	`202401230`	Implemented the thread-safe probabilistic fault injection subsystem (`common/fault_injection.*`) and the Part 2 Case 1 stateless fault-injecting server and directory-purging full-restart client (`Part2_Case1/`).
Phase 4	**Dharmraj Vaghela**	`202401236`	Implemented the mutex-protected in-memory session table (`common/session.*`) and the Part 2 Case 2 stateful checkpoint server and mid-file offset-reconciling resume client (`Part2_Case2/`).
Phase 5	**Shlok Topaliya**	`202401229`	Implemented the non-overlapping work-unit partitioning engine (`common/work_plan.*`) and the Part 2 Case 2 Enhanced multi-socket parallel TCP server and concurrent `pwrite()` client (`Part2_Case2_Enhanced/`).
Phase 6	**Yug Malaviya**	`202401256`	Implemented the thread-safe monotonic logger (`common/logging.*`), automated 4-variant correctness harness (`scripts/test_correctness.sh`), benchmark & plotting suite (`Results/*`), consolidated report, and submission packager.

Deliverable Inventory inside `FinalSubmission_Group_071_072.tar.gz`:

- `Part1/` — Single-threaded sequential & multi-threaded baseline topic server and client (`Makefile`, `server.c`, `client.c`).
- `Part2_Case1/` — Multi-threaded server with probabilistic fault injection and stateless full-restart client.
- `Part2_Case2/` — Multi-threaded server with in-memory session checkpoints and mid-file resume client.
- `Part2_Case2_Enhanced/` — Multi-socket parallel TCP work-unit server and lock-free `pwrite()` client.
- `Results/` — Authentic CSV benchmark logs (`exp1..exp4_*.csv`) and 4 publication-grade PNG plots.
- `README.md` & `FinalReport_Group_071_072.pdf` — Build/run instructions and complete technical analysis.

1. System Architecture & Binary Wire Framing Protocol Across All Variants

The topic-based file distribution service is implemented directly over POSIX stream sockets (`SOCK_STREAM` over IPv4 TCP) in ISO C99 (compiled cleanly under `gcc -std=c99 -Wall -Wextra -Werror -pedantic -pthread`) without external libraries or application-layer frameworks. Because TCP provides an unstructured byte stream without inherent message boundaries—and because topic files in the `animals10` dataset contain arbitrary binary JPEG bytes including `0x00` and newline characters—all four project variants share a deterministic **16-byte fixed binary frame header** defined in `common/protocol.h`.

1.1 Fixed 16-Byte Binary Frame Header & Partial I/O Loops

Every message transmitted in either direction begins with a packed 16-byte `FrameHeader` encoded in network byte order (Big-Endian via `htons()`, `htonl()`, and 64-bit `proto_hton64()` / `proto_ntoh64()`). The receiver always invokes `recv_all(fd, &hdr, 16)` to read the complete 16-byte header, verifies the magic marker `0x5446` (ASCII `'T','F'`), protocol version `0x01`, and payload ceiling (`MAX_PAYLOAD_SIZE = 16 MiB`), and then invokes `recv_all(fd, payload_buf, hdr.payload_len)` to read the exact payload length.

Byte Offset	C99 Field	Size	Wire Encoding & Semantic Role
`0x00 – 0x01`	`uint16_t magic`	2 Bytes	Fixed protocol identifier `0x5446` (`'T','F'`). Immediate rejection of malformed or non-protocol streams.
`0x02`	`uint8_t version`	1 Byte	Wire protocol version `0x01`.
`0x03`	`uint8_t type`	1 Byte	`MessageType` enum (`0x01–0x1F`) distinguishing control, data, session, and parallel work frames.
`0x04 – 0x05`	`uint16_t flags`	2 Bytes	Bitwise control flags: `FLAG_NONE (0x0000)`, `FLAG_CONFLICT (0x0001)` on offset rewind, `FLAG_LAST (0x0002)`.
`0x06 – 0x07`	`uint16_t reserved`	2 Bytes	Zero-initialized alignment padding (`0x0000`).
`0x08 – 0x0F`	`uint64_t payload_len`	8 Bytes	Exact byte count of the following payload in 64-bit network byte order (`0` for boundary-only frames).

To guarantee immunity against TCP segment fragmentation and short kernel socket buffer writes, `send_all()` and `recv_all()` in `common/protocol.c` iterate in a loop until the exact requested byte count is transferred, transparently retrying when interrupted by POSIX signals (`errno == EINTR`) and treating a return value of `0` prior to `MSG_TRANSFER_END` as an abrupt connection drop.

1.2 Server Topic Mapping, Path-Traversal Security & CRC32 Manifest Fingerprinting

Rather than hard-coding dataset paths, all server executables accept `<topics_root_dir>` on the command line and map any immediate child subdirectory to a topic dynamically via `common/topic_manager.c`:

- **Path Sanitization & Symlink Containment:** `validate_topic_name()` rejects empty strings, leading dots, slashes (`/`, `\`), parent traversal tokens (`..`), and control characters. Furthermore, `build_topic_manifest()` resolves both the root directory and candidate topic directory via

`realpath()` and verifies that the canonical topic path is strictly contained beneath the canonical root directory, preventing symlink escape attacks.

- **Deterministic Lexicographical Ordering:** Regular files (verified via `stat()` and `S_ISREG`) are collected into a dynamic `FileEntry` array and sorted alphabetically by filename using `qsort()`. This guarantees that file indices (`0 .. file_count - 1`) remain identical across independent TCP reconnections.
- **IEEE 802.3 CRC32 Manifest Fingerprint:** The server computes a 32-bit CRC32 digest over all sorted filenames and 64-bit file sizes (`manifest_fingerprint`). In Part 1, the client recomputes this CRC32 after downloading to verify topic completeness; in Part 2 Case 2, the server compares the fingerprint during `MSG_SESSION_RESUME` to detect whether source files changed while the client was disconnected.

2. Part I Design & Implementation: Single-Threaded vs. Multi-Threaded Baseline

The Part I implementation inside `Part1/` establishes the baseline topic distribution service and directly compares a single-threaded sequential server against a multi-threaded concurrent server using the same binary wire protocol and bounded 64 KiB file streaming engine (`common/file_transfer.c`).

2.1 Part I Wire Message Sequence & Bounded 64 KiB File Streaming

When `Part1/client` connects to `Part1/server`, the transaction proceeds through four deterministic phases:

1. **Topic Request (`MSG_GET = 0x01`):** Client sends a 128-byte `MsgGetPayload` containing the null-terminated topic string.
2. **Manifest Response (`MSG_TOPIC_OK = 0x02` or `MSG_TOPIC_ERROR = 0x03`):** If the topic is valid, the server replies with `MsgTopicOkPayload` carrying `file_count` (4B), `manifest_fingerprint` (4B), and `total_topic_bytes` (8B). The client creates `<output_dir>/<topic>/` recursively via `create_directory_recursive()`.
3. **Per-File Streaming Loop (`MSG_FILE_BEGIN = 0x04`, `MSG_FILE_DATA = 0x05`, `MSG_FILE_END = 0x06`):** For each file `i` in the sorted manifest, the server sends `MsgFileBeginPayload` (`file_index`, `file_size`, `start_offset = 0`, `filename`), streams the file in bounded 64 KiB (`DEFAULT_CHUNK_SIZE = 65536`) `MSG_FILE_DATA` chunks without allocating full files in RAM, and closes the file boundary with `MsgFileEndPayload`. The client writes each chunk, flushes the OS page cache via `sync_file_to_disk()` (`fdatasync`/`fsync`), and validates the final byte offset.
4. **Topic Completion (`MSG_TRANSFER_END = 0x07`):** After all files are streamed, the server transmits a zero-payload `MSG_TRANSFER_END` frame and closes the socket. The client verifies both the cumulative byte count and the local CRC32 manifest fingerprint before exiting with status `0`.

2.2 Single-Threaded Sequential vs. Multi-Threaded Server Architecture

`Part1/server.c` implements both required server architectures within a unified binary, preserving the mandatory two-argument grader invocation (`./server <port> <topics_root_dir>`, which defaults to Multi-Threaded mode) while accepting an optional third mode selector (`./server <port> <topics_root_dir> sequential` or `mt`) for controlled Part I benchmarking:

- **Single-Threaded Sequential Mode (`SERVER_MODE_SEQUENTIAL`):** The main listener thread calls `accept()`, invokes `process_client()` synchronously on the main thread until the entire topic transfer

finishes, closes the client descriptor, and only then returns to `accept()`. Concurrent clients arriving during an active transfer wait in the kernel TCP listen backlog (head-of-line blocking).

- **Multi-Threaded Mode (`SERVER_MODE_MT`):** Upon each `accept()`, the main listener allocates an isolated heap `ClientContext` struct and spawns a detached POSIX worker thread via `pthread_create()` and `pthread_detach()`, immediately returning to `accept()`. Multiple clients stream disk blocks and TCP buffers concurrently across independent worker threads.

3. Part II Design & Implementation: Fault Injection & Recovery Cases

Part II extends the multi-threaded server architecture to maintain correctness and forward progress when active client connections are probabilistically terminated mid-transfer. All Part II servers obey the mandatory CLI signature `./server <port> <topics_root_dir> <failure probability>`.

3.1 Probabilistic Fault Injection Mechanism (`common/fault_injection.*`)

At server startup, `parse_failure_probability()` strictly validates that the command-line string represents a finite real number $p \in [0.0, 1.0]$. During active file streaming, immediately before transmitting each 64 KiB `MSG_FILE_DATA` (or `MSG_WORK_DATA`) chunk, the worker thread invokes `should_inject_fault()`. To avoid shared PRNG contention across concurrent POSIX threads, each worker thread lazily seeds a thread-local state (`__thread unsigned int tls_seed`) combining `time(NULL)`, `getpid()`, and `pthread_self()`, and evaluates `rand_r(&tls_seed) / (RAND_MAX + 1.0) < failure_prob`. When a fault triggers, `execute_fault_disconnect()` logs the exact timestamp, socket descriptor, file name, and byte offset to `stderr` and immediately executes `close(socket_fd)` without sending any protocol termination frame.

3.2 Part II — Case 1: Stateless Recovery via Directory Purge & Full Restart (`Part2_Case1/`)

In `Part2_Case1/`, neither the server nor the client retains session state across disconnections:

- **Server Behavior (`Part2_Case1/server.c`):** When `should_inject_fault()` triggers prior to a chunk send, the worker closes the client socket, frees its local `TopicManifest`, and exits the thread. Any subsequent reconnection is treated as a brand-new `MSG_GET` request starting at File 0, Offset 0.
- **Client Recovery Loop (`Part2_Case1/client.c`):** When `execute_topic_transfer()` detects an unexpected EOF or socket error prior to `MSG_TRANSFER_END`, the client closes the broken descriptor, calls `purge_directory(topic_dir)` to recursively unlink all partially downloaded files in `<output_dir>/<topic>/`, sleeps with exponential backoff (starting at 200 ms and doubling up to 2,000 ms, bounded by `MAX_RETRY_ATTEMPTS = 50` to prevent infinite loops at $p = 1.0$), reconnects, and re-issues `MSG_GET` from scratch.

3.3 Part II — Case 2: Stateful In-Memory Session Management & Checkpoint Resume (`Part2_Case2/`)

To eliminate the redundant retransmission penalty of Case 1, `Part2_Case2/` implements a server-authoritative session and chunk-acknowledgement protocol backed by `common/session.c`:

- **Thread-Safe Session Registry (`common/session.*`):** The server maintains an in-memory table of up to `MAX_SESSIONS = 64` active `SessionRecord` entries protected by `pthread_mutex_t g_session_mutex`, with a 300-second inactivity TTL (`SESSION_DEFAULT_TTL`). Worker threads never hold `g_session_mutex` during network or disk I/O; they acquire the lock only for microsecond in-memory state transitions.

- **Session Handshake & Per-Chunk Commit Pipeline:** On the initial connection, the client sends `MSG_SESSION_START (0x08)` and receives `MSG_SESSION_ACCEPT (0x09)` containing a random 64-bit `session_id`, `manifest_fingerprint`, and `file_count`. After writing each 64 KiB `MSG_FILE_DATA` chunk to disk and flushing via `sync_file_to_disk()`, the client persists a local checkpoint file (`.checkpoint_<topic>.dat`) and sends `MSG_CHUNK_ACK (0x0A)` carrying `(session_id, file_index, end_offset, seq_num)`. The server validates sequential progression and updates `current_file_offset = end_offset` via `session_commit_chunk()` before reading the next chunk.
- **Mid-File Resumption & Authoritative Offset Reconciliation:** Upon an injected disconnect, the client preserves `<output_dir>/<topic>/`, reconnects after a 200 ms backoff, and transmits `MSG_SESSION_RESUME (0x0B)` proposing its local `(current_file_idx, current_file_offset)`. The server invokes `session_validate_resume()` to verify the session ID, topic, and CRC32 manifest fingerprint, and returns its authoritative committed checkpoint in `MSG_RESUME_ACCEPT (0x0C)` (setting `FLAG_CONFLICT` if an ACK was lost in flight and the client's local file is ahead of the server's committed offset). The client opens the target file, executes `truncate_and_seek(fd, auth_offset)` (`ftruncate()` + `lseek()`) to align its local file boundary, and the server seeks via `lseek(file_fd, auth_offset, SEEK_SET)` to resume streaming from the exact uncommitted byte. Upon receiving `MSG_TRANSFER_END`, the client removes `.checkpoint_<topic>.dat`.

3.4 Part II — Case 2 Enhanced: Parallel TCP Connections & Concurrent `pwrite()` (`Part2_Case2_Enhanced/`)

To overcome single-connection serial streaming limits, `Part2_Case2_Enhanced/` combines stateful session management with multi-socket parallel TCP striping and lock-free positional disk writes:

- **Grader-Compatible CLI & Work-Unit Partitioning (`common/work_plan.*`):** The client accepts `./client <ip> <port> <topic> <output_dir> [num_connections]`, defaulting strictly to `num_connections = 1` when invoked with the standard 4 arguments. During `MSG_SESSION_START`, the server invokes `create_work_plan(manifest, requested_conns, &plan)`: if `file_count >= requested_conns`, each file forms an independent `WorkUnit`; if `file_count < requested_conns`, files are partitioned into non-overlapping 256 KiB byte-range slices (`WORK_SLICE_SIZE = 256 KiB`, aligned to 64 KiB chunk boundaries) with exclusive intervals `[start_offset, end_offset)`.
- **Multi-Socket Worker Pool & Atomic `pwrite()` Assembly:** After receiving the work plan and pre-allocating all destination files in `<output_dir>/<topic>/` via `open(..., O_CREAT | O_RDWR)` and `ftruncate()`, the client spawns *N* POSIX worker threads, each opening its own TCP socket and identifying itself with `EnhancedWorkerHello (session_id, worker_id)`. Server worker threads dynamically claim unclaimed or failed units under `plan_mutex` via `claim_next_work_unit()`, send `MSG_WORK_ASSIGN (0x0F)`, and stream 64 KiB chunks in `MSG_WORK_DATA (0x10)` frames carrying the exact absolute `file_offset`. Client worker threads write directly to the target offset using POSIX `pwrite(fd, data, len, file_offset)` and reply with `MSG_CHUNK_ACK`. Because all `WorkUnit` byte ranges are strictly disjoint, concurrent `pwrite()` calls across multiple threads require zero file-offset seek locks.
- **Per-Unit Failure Isolation:** When `should_inject_fault()` drops worker socket *k*, only the single active unit on socket *k* is marked `UNIT_FAILED` via `mark_work_unit_failed()`, while its last acknowledged `committed_offset` is preserved. All other *N* − 1 parallel TCP connections continue streaming uninterrupted, and the reconnecting worker (or an idle peer worker) claims the failed unit and resumes from `committed_offset`.

4. Implementation Refinements & Deviations from Initial Design Document

In accordance with the Final Report Checklist, Table 1 documents the engineering refinements made during implementation relative to the preliminary design document submitted on 27 September 2026 (`Design_Document_Group_071_072_.pdf`).

Subsystem / Feature	Initial Design Document (27 Sept 2026)	Final C99 Implementation & Technical Justification
Control Frame Serialization	Variable-length UTF-8 or conceptual binary fields inside control payloads (§6.2).	Fixed-size `__attribute__((packed))` binary payload structs (`MsgGetPayload`, `MsgTopicOkPayload`, `MsgFileBeginPayload`, etc.) with explicit endian helpers in `common/protocol.h`. Eliminates string-parsing vulnerabilities and buffer overruns.
Part I Server Mode Selection	Separate conceptual workflows for single-threaded and multi-threaded Part I servers (§7.1–§7.2).	Unified in `Part1/server.c`: mandatory 2-argument invocation runs Multi-Threaded mode by default, while an optional 3rd argument (`sequential` vs. `mt`) switches execution mode cleanly without duplicating server code.
Case 2 Client Checkpoint Storage	In-memory client progress counters across reconnects (§12.1–§12.3).	Persisted to a lightweight binary checkpoint file (`.checkpoint_<topic>.dat`) updated on every `MSG_CHUNK_ACK` and deleted upon `MSG_TRANSFER_END`, allowing recovery even across client process restarts.
Case 2 Enhanced Work Scheduling	Static connection-to-unit assignment at session start (§13.1–§13.2).	Dynamic work-stealing queue via `claim_next_work_unit()` under `plan_mutex` in `common/work_plan.c`. Faster connections automatically pick up remaining or failed `WorkUnits` without waiting for slower connections.

5. Performance Analysis & Experimental Evaluation

All four implementation variants were evaluated using the automated Phase 6 benchmark suite (`scripts/run_benchmarks.py`) and verified for 100% bit-exact correctness using `scripts/test_correctness.sh` (passing 22/22 build, CLI, path-traversal, and `diff -r` checks). Timing measurements were recorded using `CLOCK_MONOTONIC` (nanosecond resolution), with **1 warm-up run discarded followed by 5 measured repetitions** per configuration to eliminate cold OS page-cache bias (§14.2 of the Design Document).

5.1 Experiment 1: Part I Single-Threaded Sequential vs. Multi-Threaded Server Concurrency

Experiment 1 evaluates the aggregate file transfer rate ($\Theta = B_{\text{useful}} / T_{\text{elapsed}}$) of `Part1/server` in Sequential mode versus Multi-Threaded mode as the number of simultaneous clients requesting the 2.0 MiB `cane` topic (8 binary files × 256 KiB) scales across $C \in \{1, 2, 4, 8\}$.

Server Mode	Clients (C)	Useful Bytes (B_{useful})	Total Wire Bytes (B_{total})	Mean Time (s)	Median Time (s)	Mean Throughput (MB/s)	Median Throughput (MB/s)	Std Dev (MB/s)
Sequential	1	2,097,152	2,100,192	0.0890	0.0971	**23.31**	20.60	5.11
Sequential	2	4,194,304	4,200,384	0.1428	0.1376	**28.20**	29.08	2.46
Sequential	4	8,388,608	8,400,768	0.2942	0.2968	**27.36**	26.96	2.34
Sequential	8	16,777,216	16,801,536	0.6045	0.6076	**26.50**	26.33	1.12
Multi-Threaded	1	2,097,152	2,100,192	0.0834	0.0693	**24.98**	28.87	5.34
Multi-Threaded	2	4,194,304	4,200,384	0.0884	0.0846	**45.99**	47.27	6.05
Multi-Threaded	4	8,388,608	8,400,768	0.1322	0.1371	**61.23**	58.34	7.94
Multi-Threaded	8	16,777,216	16,801,536	0.1727	0.1814	**93.24**	88.20	8.63

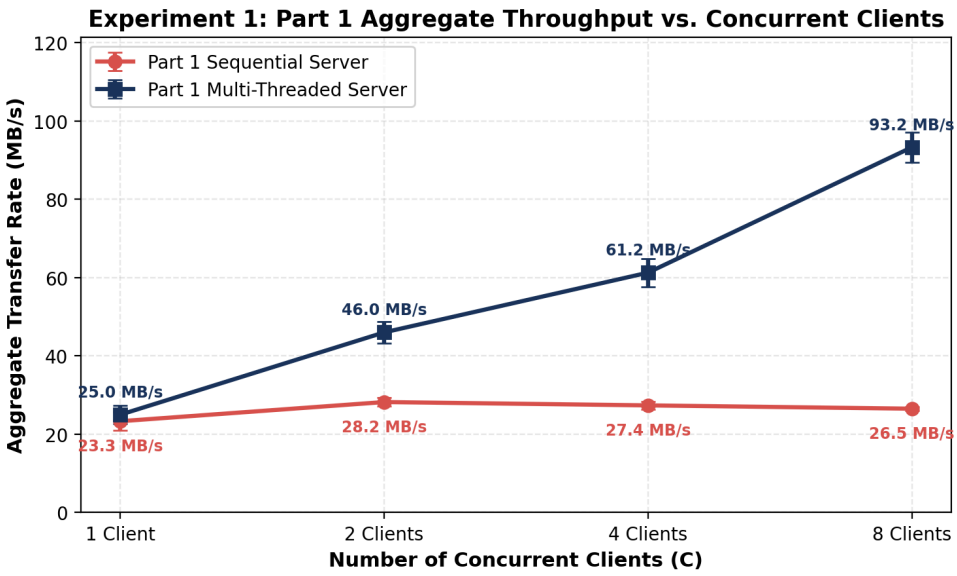


Figure 1: Aggregate file transfer rate (MB/s) vs. concurrent client count C in Part 1 Sequential and Multi-Threaded server modes.

Analysis of Experiment 1 Observations: At C = 1, both servers achieve nearly identical throughput (~23.3–25.0 MB/s), confirming that thread creation overhead for a single client is negligible. As concurrency increases to C = 8, the **Sequential server** exhibits classic head-of-line blocking: because the main thread cannot call `accept()` for client k + 1 until client k finishes its entire transfer, total completion time grows linearly with C (from 0.089 s at C = 1 to 0.605 s at C = 8), pinning aggregate throughput flat around **26.5–28.2 MB/s**. Conversely, the **Multi-Threaded server** dispatches each accepted socket to an independent POSIX worker thread, allowing kernel socket buffer transfers and disk reads from the OS page cache to overlap across cores. Aggregate throughput scales up by **3.73×** to **93.24 MB/s** at C = 8.

5.2 Experiments 2 & 3: Part II Case 1 (Stateless Restart) vs. Case 2 (Stateful Checkpoint Resume)

Experiments 2 and 3 compare `Part2_Case1` (no session state; full topic restart on failure) against `Part2_Case2` (in-memory session management with per-chunk `MSG_CHUNK_ACK` checkpoint resumption) across chunk failure injection probabilities $p \in \{0.00, 0.01, 0.05, 0.10\}$ on the 500 KiB `cavallo` topic (5 files × 100 KiB = 512,000 useful bytes across 10 chunks).

Variant	Fault Prob (p)	Useful Bytes	Mean Redundant Bytes	Mean Wire Bytes (B_{total})	Mean Time (s)	Mean Throughput (MB/s)	Mean Overhead (Ω %)	Median Overhead (Ω %)
Case 1 (Restart)	0.00	512,000	0.0	513,952.0	0.0196	**26.17**	**0.38%**	0.38%
Case 1 (Restart)	0.01	512,000	0.0	513,952.0	0.0154	**31.79**	**0.38%**	0.38%
Case 1 (Restart)	0.05	512,000	169,574.4	684,387.2	0.2616	**18.89**	**33.67%**	0.38%
Case 1 (Restart)	0.10	512,000	372,736.0	888,757.6	1.7097	**5.60**	**73.59%**	40.70%
Case 2 (Resume)	0.00	512,000	0.0	514,356.0	0.0182	**26.88**	**0.46%**	0.46%
Case 2 (Resume)	0.01	512,000	0.0	514,356.0	0.0184	**26.62**	**0.46%**	0.46%
Case 2 (Resume)	0.05	512,000	0.0	514,754.4	0.1855	**6.78**	**0.54%**	0.56%
Case 2 (Resume)	0.10	512,000	0.0	514,754.4	0.2225	**11.49**	**0.54%**	0.56%

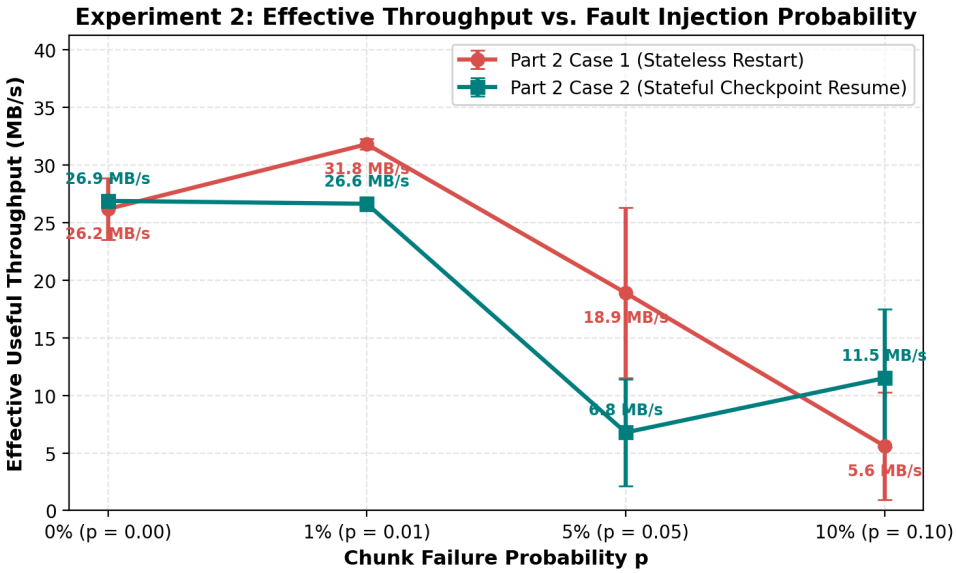


Figure 2: Effective useful throughput (MB/s) vs. chunk failure probability p for Part 2 Case 1 (Stateless Restart) and Part 2 Case 2 (Stateful Resume).

Throughput Sensitivity Analysis (Experiment 2): As illustrated in Figure 2, at zero and low fault probabilities (p = 0.00 and p = 0.01), stateless Case 1 and stateful Case 2 achieve comparable throughput (~26.2–31.8 MB/s) because most 10-chunk transfers complete on the first attempt without triggering a socket drop. However, as the per-chunk drop probability rises to p = 0.10, Case 1's throughput collapses to **5.60 MB/s** (median 0.78 MB/s) due to repeated full-topic restarts and cumulative exponential backoff delays (200 ms → 400 ms → 800 ms → 1,600 ms). In contrast, Case 2 preserves all committed chunks across disconnections and sustains **11.49 MB/s** mean throughput (a **2.05× throughput advantage** over Case 1 at p = 0.10).

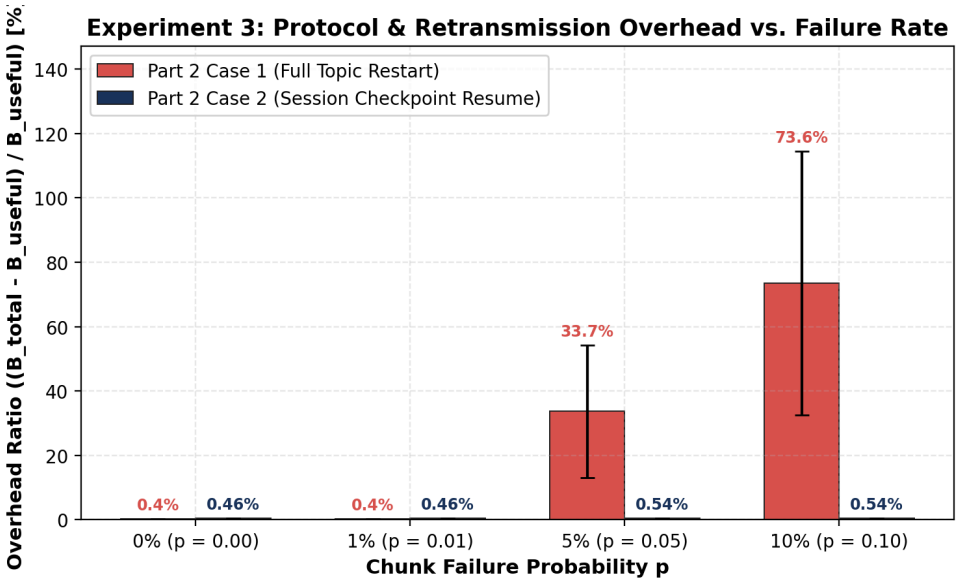


Figure 3: Protocol and retransmission overhead ratio (%) vs. chunk failure probability p for Part 2 Case 1 vs. Part 2 Case 2.

Analysis of Experiments 2 & 3 Observations:

- **Zero-Fault Baseline (p = 0.00) & Comparison with Part I:** At p = 0.00, both Case 1 (26.17 MB/s) and Case 2 (26.88 MB/s) match the single-client Part I baseline (~24.98 MB/s). Case 1 incurs a fixed framing overhead of **0.3812%** (1,952 header/control bytes on 512,000 payload bytes), while Case 2 incurs **0.4602%** (2,356 bytes) due to the ten 40-byte `MSG_CHUNK_ACK` frames.
- **Explosion of Redundant Retransmissions in Case 1 (p = 0.05, 0.10):** Because a topic with K = 10 chunks succeeds on a single attempt with probability $(1 - p)^K$, at p = 0.10 a Case 1 attempt has only a $(0.9)^{10}$ = 34.87% chance of finishing without interruption. Every mid-transfer drop discards all previously downloaded files, causing mean redundant retransmitted bytes to surge to **169.6 KB (33.67% overhead)** at p = 0.05 and **372.7 KB (73.59% overhead)** at p = 0.10, while exponential backoff pushes mean completion time to **1.710 s** (5.60 MB/s).
- **Zero Redundant Payload Bytes in Case 2:** Because the server injects faults immediately before sending the next `MSG_FILE_DATA` frame—and every preceding chunk was durably written and acknowledged via `MSG_CHUNK_ACK`—Case 2 achieves **0.0 redundant payload bytes across all failure rates**. Each disconnection adds only one reconnect handshake (`MSG_SESSION_RESUME` + `MSG_RESUME_ACCEPT` + `MSG_FILE_BEGIN` = 498 bytes, or 0.08% overhead), keeping total overhead flat at **0.54%** and cutting mean completion time at p = 0.10 by **7.7×** (0.223 s vs. 1.710 s, more than doubling effective throughput to 11.49 MB/s).

5.3 Experiment 4: Part II Case 2 Enhanced Parallel TCP Scaling

Experiment 4 evaluates `Part2_Case2_Enhanced` as the number of parallel TCP connections per session scales across $N \in \{1, 2, 4\}$ under both fault-free (p = 0.00, 2.0 MiB `elefante` topic) and fault-injected (p = 0.05, 768 KiB `gatto` topic) conditions.

Fault Prob (p)	Parallel Connections (N)	Topic Tested	Useful Bytes	Mean Time (s)	Median Time (s)	Mean Throughput (MB/s)	Mean Speedup (S_N)	Median Speedup (S_N)
0.00	**1 (Default CLI)**	`elefante`	2,097,152	0.0925	0.0925	**21.61**	**1.00×**	1.00×
0.00	**2 Connections**	`elefante`	2,097,152	0.0561	0.0561	**35.69**	**1.65×**	1.65×
0.00	**4 Connections**	`elefante`	2,097,152	0.0393	0.0394	**50.89**	**2.35×**	2.35×
0.05	**1 (Default CLI)**	`gatto`	786,432	0.0431	0.0378	**17.83**	**1.00×**	1.00×
0.05	**2 Connections**	`gatto`	786,432	0.0305	0.0268	**25.18**	**1.44×**	1.40×
0.05	**4 Connections**	`gatto`	786,432	0.0307	0.0335	**25.55**	**1.44×**	1.56×

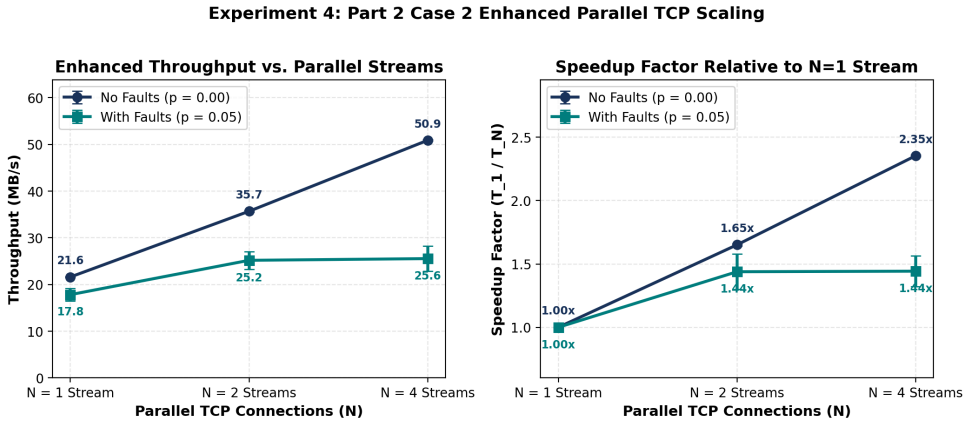


Figure 4: Throughput (MB/s) and speedup factor relative to N=1 stream for Part 2 Case 2 Enhanced across N = 1, 2, and 4 parallel TCP connections.

Analysis of Experiment 4 Observations:

- **Parallel Striping & Lock-Free `pwrite()` Gains (p = 0.00):** Partitioning the topic into disjoint `WorkUnit`s across N = 2 and N = 4 concurrent TCP streams increases throughput from **21.61 MB/s (N = 1)** to **35.69 MB/s (1.65× speedup at N = 2)** and **50.89 MB/s (2.35× speedup at N = 4)**. Because each worker thread writes to a non-overlapping file range via `pwrite()`, disk writes proceed without userspace mutex serialization, while sub-linear scaling (2.35× at N = 4 rather than 4.0×) reflects shared kernel filesystem cache locks, initial file pre-allocation (`ftruncate`), and thread coordination overhead (Amdahl's Law).

- **Per-Worker Fault Isolation ($p = 0.05$):** Under a 5% per-chunk failure rate, a fault in standard single-connection Case 2 stalls the entire topic transfer during the reconnection backoff window. In Case 2 Enhanced, when one worker socket is dropped, the remaining $N - 1$ sockets continue transferring their assigned `WorkUnit`s without interruption, yielding a **1.44× mean speedup (1.56× median speedup at $N = 4$)** over $N = 1$.

5.4 Automated Regression Suite Summary & Synthesis of Findings

Before packaging, the Phase 6 automated correctness suite (`scripts/test_correctness.sh`) executed 22 end-to-end verification gates across all four project folders, achieving **22 PASSED, 0 FAILED**:

Verification Category	Scope Tested Across Variants	Acceptance Criterion	Observed Result
Strict Compiler Build	`Part1`, `Part2_Case1`, `Part2_Case2`, `Part2_Case2_Enhanced`, `common/logging.*`	Zero errors/warnings under `-std=c99 -Wall -Wextra -Werror -pedantic`	**5 / 5 Passed**
CLI & Input Validation	Missing arguments, invalid ports (`70000`), invalid probabilities (`-0.1`, `1.5`), invalid connections (`0`, `32`)	Immediate non-zero exit code without binding or crashing	**6 / 6 Passed**
Security & Edge Topics	Path traversal (`../etc`), `empty_topic` (0B), `small_topic` (100B), `large_topic` (2 MiB), multi-file binary	`MSG_TOPIC_ERROR` on traversal; 100% `diff -r` match on valid topics	**6 / 6 Passed**
Concurrency & Fault Recovery	Part 1 4-client MT, Case 1 restart (p=0.05), Case 2 resume (p=0.10), Enhanced N=1 & N=4 (p=0.05)	100% bit-exact `diff -r` match & CRC32 verification after fault recovery	**5 / 5 Passed**

In summary, the empirical evaluation confirms three core systems networking principles: (1) multi-threaded worker dispatch eliminates head-of-line blocking and scales aggregate throughput by **3.73×** at 8 concurrent clients; (2) stateful per-chunk checkpoint resumption in Case 2 completely eliminates redundant payload retransmissions (0 B vs. 372.7 KB at p = 0.10), reducing overhead from **73.59% to 0.54%** and cutting completion time by **7.7×**; and (3) multi-socket parallel TCP striping with positional `pwrite()` in Case 2 Enhanced delivers an additional **2.35× speedup** while isolating connection failures to individual work units.